

Instead of `i**2` here we could be calling a complex function and building an array from the result of that.

Like with most programming languages you can access elements of an the list using square brackets. Like what we discussed with strings, it's possible to access sequences of elements of arrays, to skip elements and to reverse elements using `array[start:stop:stride]`. It's also possible to assign elements this way. So if you wanted to set the second and third elements of an list to 5, and 7 respectively you can do:

- `array[1:3] = [5, 7]`

Tuples

Tuples are quite similar to lists, in that they are a sequence of values. However where lists are intended to be modified once created, tuples are sequences that cannot be changed once created. While the use of this might not be readily apparent to you it's important to know that they exist and how to use them because they occur quite commonly in Python.

A tuple can be created the same way as lists but by using round brackets instead of square ones. For instance:

- `position = (5, 7)`

One immediate use of this is to return multiple values from functions.

Dictionaries

Python dictionaries or dicts allow you to create associative arrays / hashes that can have arbitrary (though constant) values as indices instead of numbers. For instance if we are processing some text and want to count the number of times a particular word occurs we can use dicts as follows:

- `d = {}`
- `d['the'] = 100`
- `d['a'] = 125`
- `d['ophthalmologist'] = 1`

Note that while we have used strings here, the index of a dict could also be a number or even a tuple!

Another way to define the above would be:

- `d = {`
- `'the': 100,`
- `'a': 125,`
- `'ophthalmologist': 1`
- `}`